Valkey Feature Store

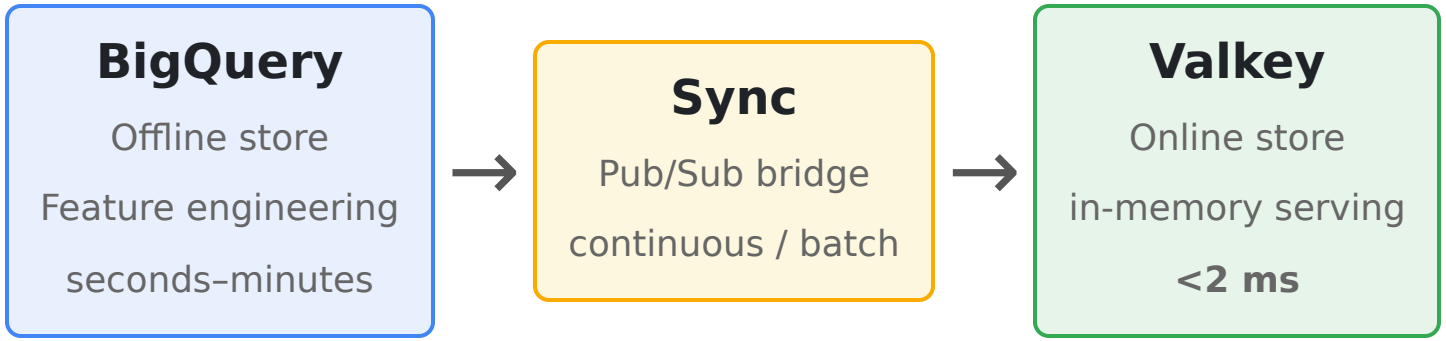
9 notebooks — sub-millisecond in-memory serving

Memorystore for Valkey | HNSW vector search | atomic real-time features

github.com/statmike/vertex-ai-mlops/MLOps/Feature Store/Valkey

The Feature Store Problem

Features are computed in batch (BigQuery) but served in real time. The online store sits between them — and when **microseconds matter** (trading, gaming, ad bidding), it has to be in memory.



This series builds a complete feature store on **Memorystore for Valkey** — using BigQuery as the offline engine and Valkey for ultra-low-latency online serving.

Why Valkey? (Redis vs Valkey on Memorystore)

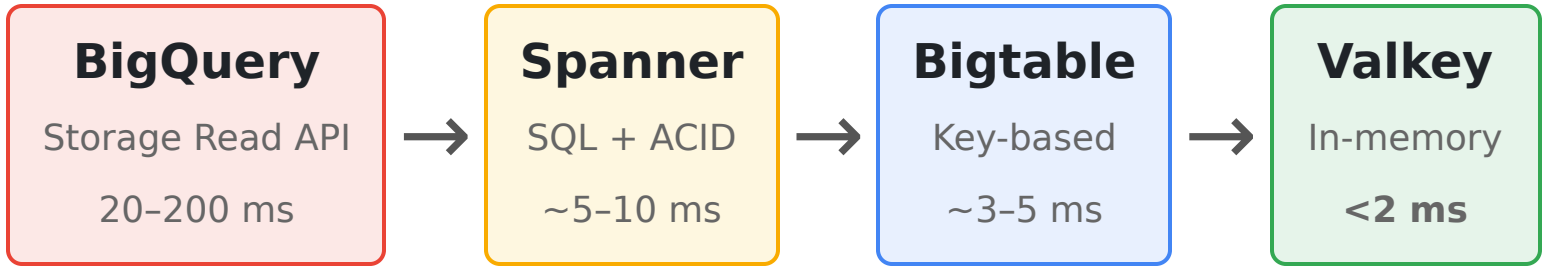
Google Cloud offers two managed in-memory stores. This series uses **Valkey** — the patterns work on either, since Valkey is Redis-protocol compatible.

	Memorystore for Redis	**Memorystore for Valkey**
Engine	Redis, up to 7.2	Valkey 7.2 / 8.0 / 9.0
Governance	Redis Inc. (SSPL/RSAL)	**Open source, Linux Foundation (BSD)**
Architecture	Primary + replica	**Cluster-native** (+ Cluster-Disabled option)
Scale-out	Vertical	**Horizontal** (add shards)
Newer features	Capped at 7.2	HEXPIRE, ongoing OSS development

We run **Cluster Mode Disabled** — single endpoint, standard `redis.Redis` client, all multi-key ops work without slot constraints.

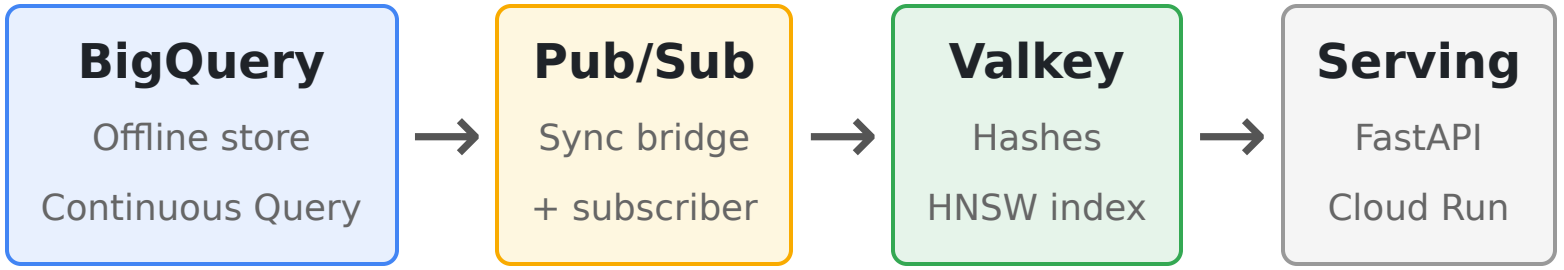
The Latency Spectrum

Where Valkey sits among the five Google Cloud feature store approaches:



Valkey gives the **fastest reads in the series** plus native vector search. The trade-off: data lives in memory, sync is a custom Pub/Sub bridge, and it's a serving layer — BigQuery remains the durable source of truth.

Architecture



Features are stored as **Redis hashes**, key `features:{entity_group}:{entity_id}`. Applications also write real-time signals directly (dual-write path).

Concept	Valkey	Bigtable equivalent
Hash key	`features:A:000001`	Row key `A#000001`
Hash field	`feature_float_1`	Column qualifier
Read all	`HGETALL`	ReadRow
Atomic add	`HINCRBY`	ReadModifyWriteRow

The Data

A **130K-entity dataset** shared across all feature store series — 26 entity groups (A–Z), 5,000 entities per group, 224 columns spanning every BigQuery data type.

Resource	Size	Purpose
`dense_features`	130,000 rows, 224 cols	Every BQ data type
`sparse_events`	2,600,000 rows	Time-series events
`valkey_feature_store`	BigQuery dataset	Independent per-series dataset

Loaded into Valkey via pipelined `HSET`: **130K entities in ~25s**, ~300 MB memory.

Fundamentals — Hashes & Sub-ms Reads

Read patterns

- `HGET` — single field
- `HMGET` — selected fields
- `HGETALL` — whole entity

Measured (130K entities)

Operation	Latency
Point read (HGETALL)	~1.6 ms
HMGET (subset)	~1.5 ms

Pipelining — batch N reads in one roundtrip

Entities	Total	Per-entity
1	1.3 ms	1.3 ms
10	3.0 ms	0.30 ms
100	14.6 ms	0.15 ms
500	60 ms	0.12 ms

Pipelining amortizes the network roundtrip — the key to bulk feature reads.

Serialization — Five Encoding Methods

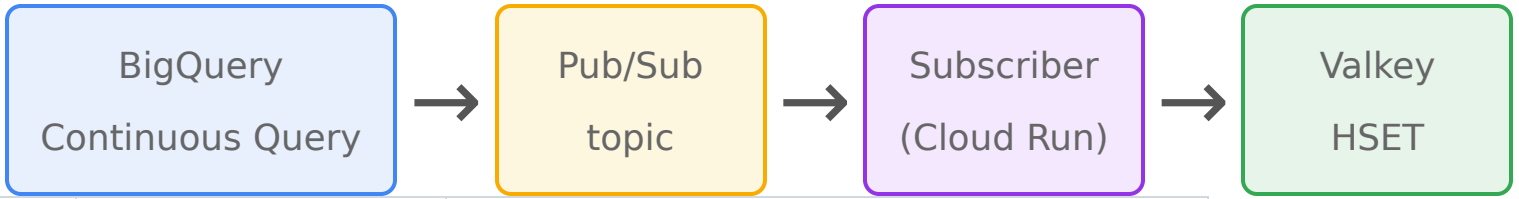
Encoding BigQuery's 19 data types into Redis strings/bytes. Five methods, each a real trade-off:

Method	Storage	Single-field read	All-field read
Native hash fields	Highest	**Fastest** (HGET)	Slowest (HGETALL)
JSON blob	Large	Slow (parse all)	Medium
Concatenation	**Smallest**	Slow (split all)	Medium
Protocol Buffers	Small	Slow (deserialize)	Medium
Hybrid (hot + cold blob)	High	**Fastest** for hot	**Fastest** overall

The Redis-specific insight: **native fields let you `HGET` one feature** — blob methods force reading the whole entity to read anything.

Synchronization — Four Patterns

No native BigQuery export to Valkey — so we build a **Pub/Sub bridge** (the reusable pattern for any target).



Pattern	Freshness	Use case
One-time backfill	Manual	Initial load (~10K entities/s)
Scheduled sync	Minutes–hours	Periodic feature refresh
Continuous query	Seconds	Near-real-time propagation
Dual-write	<1 ms	Real-time signals (app → Valkey)

TTL-on-Sync — Automatic Staleness Bounds

A pattern **unique to Valkey**: write features with an `EXPIRE`, so stale data self-evicts if the sync pipeline stalls.

Set TTL = sync interval × safety factor. A missing key means "stale" — the serving layer falls back to BigQuery and re-populates. An automatic staleness guarantee the disk-based stores don't have.

TTL, Eviction & Memory

In-memory means a hard ceiling — eviction is a first-class concern (Bigtable/Spanner scale disk instead).

TTL patterns

- Session features → 30 min
- Real-time signals → 5 min
- Daily aggregates → 24 hr
- Core features → permanent

Eviction: `volatile-lru` drops TTL keys first, keeps permanent features.

Operational notes

- Active vs lazy expiration: expired keys hold memory until collected
- Persistence: RDB/AOF + HA replica (BigQuery is the source of truth)
- **HEXPIRE** (per-field TTL) exists in Valkey 8.0 but Memorystore blocks it

Atomic Operations & Real-Time Features

Valkey's atomic ops are the fastest in the series — `HINCRBY` <1 ms.

Primitives

- `HINCRBY` / `HINCRBYFLOAT` — counters
- `MULTI/EXEC` — multi-key atomic
- `WATCH` — optimistic locking
- `SETNX` — idempotent write-once
- **Lua (EVAL)** — read-check-write in one server call
- **Streams (XADD)** — event log in-store

Three dynamic-feature patterns

Pattern	Best for
Read-modify-write	running totals
Read + compute	cheap derived
Streaming aggregation	windowed aggs

Combine batch features (synced) + real-time counters (direct) in one hash.

Serving Integration

FastAPI + connection pool + the correctness checks every feature store needs.

Serving path

- `HMGET` → decode → predict
- Connection pool (created once)
- Retry + timeout + graceful degradation
- Fallback to BigQuery on miss

Measured: ~6.5 ms read+predict; ~1000 RPS at 50 workers

Correctness

- **Training-serving skew** check (0% mismatch)
- Read-method comparison (HMGET ≈ Lua ≈ pipeline < HGETALL)
- Live schema evolution — add a feature with no downtime

Vector Search with HNSW

Valkey's signature ML feature: native approximate nearest neighbor via `FT.SEARCH`.

Index types

- `FLAT` — exact brute-force
- `HNSW` — approximate, sub-5 ms

Measured (130K vectors, 64-dim)

Index	Latency	Recall@10
FLAT	~7 ms	100%
HNSW	~2 ms	~89%

Hybrid filtered search

```
(@category:{electronics} @score:[50 +inf]) =>[KNN 10 @embedding $q]
```

Metadata filter **+** vector rank in one query — Bigtable can't pre-filter.

Tune recall via `M`, `EF_CONSTRUCTION`, `EF_RUNTIME` (set at index creation on Memorystore).

Vector Search — Across the Series

All four serving stores now have **native** vector search — they differ on index, pre-filtering, and latency.

Approach	Index	Pre-filtering	Latency (130K)
Valkey	HNSW / FLAT	TAG + NUMERIC	**~2–5 ms**
Spanner	ScaNN ANN / exact	**Full SQL WHERE**	~10–50 ms
BigQuery	IVF / TreeAH	SQL pushdown	~100 ms–s
Bigtable	brute-force	row key only	~100–300 ms

Valkey for sub-ms real-time retrieval. **Spanner** when vector search is one step inside a richer SQL query (filter many columns + JOIN).

Capstone — Real-Time Trading

A trading feature store exercising every Valkey capability where microseconds matter.

- **Instrument features** — hashes, sub-ms reads
- **Market quotes** — 5-second TTL (stale prices are dangerous)
- **Atomic P&L** — `HINCRBYFLOAT` per trade (~2 ms)
- **Top movers** — sorted sets (`ZADD`)
- **Instrument matching** — HNSW by factor exposure (<5 ms)
- **Two-stage recommendation** — HNSW retrieve → score (~5 ms total)
- Load tested under concurrency

Production Operations

Deploy — Terraform (`google_memorystore_instance`), PSC service connection policy, Pub/Sub with dead-letter

Monitor — Cloud Monitoring: memory usage ratio, cache hit ratio, evictions, failover

Secure — TLS + AUTH, least-privilege IAM, private (PSC) networking

HA — Standard tier: primary + replica, automatic failover, stable endpoint

Connect — Cluster Disabled → `redis.Redis`; Cluster Enabled → `RedisCluster`

Readiness checklist — sizing, eviction policy, sync, skew checks, alerts

When to Use Valkey

Sub-millisecond reads required (gaming, trading, ad bidding)? → Valkey
Large-scale real-time vector retrieval (HNSW ANN)? → Valkey
Atomic real-time counters / ephemeral session state? → Valkey
Data fits in memory and BigQuery is the source of truth? → Valkey

Capability	Valkey	Bigtable	Spanner
Point-lookup latency	<2 ms	~3–5 ms	~5–10 ms
Vector search	HNSW (native)	brute-force	ScaNN (native)
Atomic ops	<1 ms (HINCRBY)	~3 ms	txn ~5–15 ms
Storage	in-memory	disk (petabytes)	disk (auto-scale)
BQ sync	Pub/Sub bridge	EXPORT DATA	continuous query

Resources

Repository: github.com/statmike/vertex-ai-mlops/MLOps/Feature Store/Valkey

Memorystore for Valkey: cloud.google.com/memorystore/docs/valkey

Vector search: cloud.google.com/memorystore/docs/valkey/about-vector-search

Valkey: valkey.io